

Programming Language Constructs Introduction

Peter Sestoft
Niels Hallenberg
2026-07-19



Plan for today

- Course contents, goals and motivation
- Representing programs
 - Abstract syntax
 - Using F# algebraic datatypes
 - Using Java/C# class hierarchies and composites
- Manipulating abstract syntax

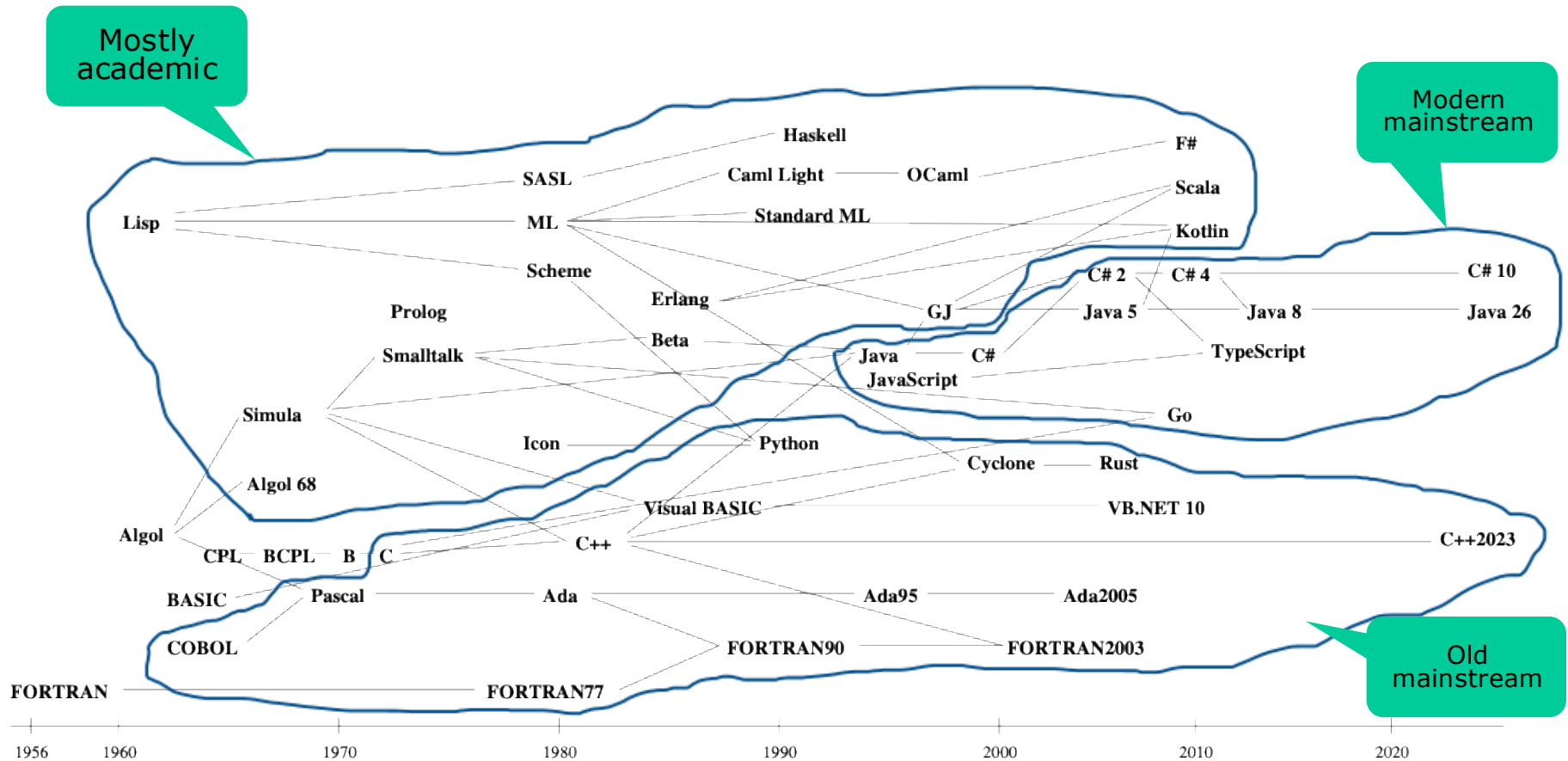
Course contents

- Abstract syntax, representing programs as data
- Lexical analysis, regular expressions, finite automata, NFA, DFA, lexer generators
- Syntax analysis, top-down versus bottom-up parsing, LL versus LR, parser generators
- Expression evaluation, stack machines, Postscript
- Compilation of a subset of C with *p, &x, pointer arithmetic's, arrays
- Type checking, type inference, statically and dynamically typed languages
- The machine model of Java, C#, F#: stack, heap, garbage collection
- The intermediate bytecode languages of the Java Virtual Machine and .NET CLR
- Garbage collection techniques, dynamic memory management
- Continuations, exceptions, a language with backtracking (an Icon subset)
- Interpretation and Compilation of a subset of Standard ML with higher order functions and exceptions.
- Type checking and Compilation of a subset of Java.
- Compilation to native ARM64 assembly language.

Sources

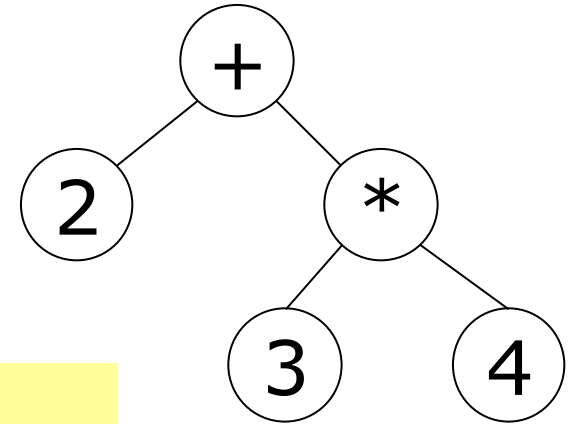
- Sestoft: *Programming Language Concepts (PLC)*, 2026
 - Buy it at Academic Books or online
- Materials on F#, such as Hansen+Rischel: *Functional programming with F#* (2013)
 - For reference, partially covered by PLC Appendix A
- Mogensen: *Introduction to Compiler Design* (ICD 2011)
or 2010 online version *Basics of Compiler Design*
 - We only use chapters 1 and 2 (2011) **or** 2 and 3 (2010)
- GitHub repository with source code
<https://github.com/sestoft/ProgrammingLanguageConcepts/>

Genealogy of some Programming Languages.



Representing abstract syntax in F#

- Think of an expression "2+3*4" as a tree
- We can represent trees using datatypes:



```
type expr =  
    | CstI of int  
    | Prim of string * expr * expr
```

```
Prim("+", CstI 2, Prim("*", CstI 3, CstI 4))
```

```
CstI 17
```

```
Prim("-", CstI 3, CstI 4)
```

```
Prim("+", Prim("*", CstI 7, CstI 9), CstI 10)
```

What
expressions?

How represent 6*0? (2+3)*4? 5+6+7? 8-9-10?

Evaluating expressions in F#

- Evaluation is a function from `expr` to `int`
- To evaluate a constant, return it
- To evaluate an operation (+, -, *)
 - evaluate its operands to get their values
 - use these values to find value of operation

RECURSION

```
let rec eval (e : expr) : int =  
    match e with  
    | CstI i -> i  
    | Prim("+", e1, e2) -> eval e1 + eval e2  
    | Prim("*", e1, e2) -> eval e1 * eval e2  
    | Prim("-", e1, e2) -> eval e1 - eval e2  
    | Prim _ -> failwith "unknown primitive";;
```

```
eval (Prim("-", CstI 3, CstI 4));;
```

Let's change the meaning of minus

- Type `expr` is the *syntax* of expressions
- Function `eval` is the *semantics* of expressions
- We can change both as we like
- Let's define subtraction so it never gives a negative result:

```
let rec evalm (e : expr) : int =  
  match e with  
  | CstI i -> i  
  | Prim("+", e1, e2) -> evalm e1 + evalm e2  
  | Prim("*", e1, e2) -> evalm e1 * evalm e2  
  | Prim("-", e1, e2) ->  
    let res = evalm e1 - evalm e2  
    if res < 0 then 0 else res  
  | Prim _ -> failwith "unknown primitive";;
```


How convert expression to a string?

- We want a function like this:

```
let rec fmt (e : expr) : string =  
  ...
```

What goes
here?

- For instance,
 `fmt (CstI 654)` gives `"654"`
 `fmt (Prim("-", CstI 3, CstI 4))` gives `"(3-4)"`
- See PLC exercise 1.2.

Expressions with variables

- Extend the `expr` type with a variable case:

```
type expr =  
  | CstI of int  
  | Var of string  
  | Prim of string * expr * expr;;
```

```
CstI 17  
Prim("+", CstI 3, Var "a")  
Prim("+", Prim("*", Var "b", CstI 9), Var "a")
```

- We need to extend the `eval` function also

```
let rec eval e : int =  
  match e with  
    | CstI i           -> i  
    | Var x            -> ???  
    | Prim("+", e1, e2) -> ...
```

How can we know the variable's value?

Use an environment

- An environment maps a name to its value
 - It is a simple dictionary or map
- Here use a list of pairs of name and value:

```
let env = [("a", 3); ("c", 78); ("baf", 666); ("b", 111)]
```

- How to look up a name in the environment:

```
let rec lookup env x =  
  match env with  
  | []          -> failwith (x + " not found")  
  | (y, v)::r   -> if x=y then v else lookup r x;;
```

- We can extend the environment by adding to the front of the list.
- What does `lookup env' "a"` return?

```
let env' = ("a", 42)::env
```

Evaluation in an environment

- The environment is an extra argument
- Must pass the environment in recursive calls

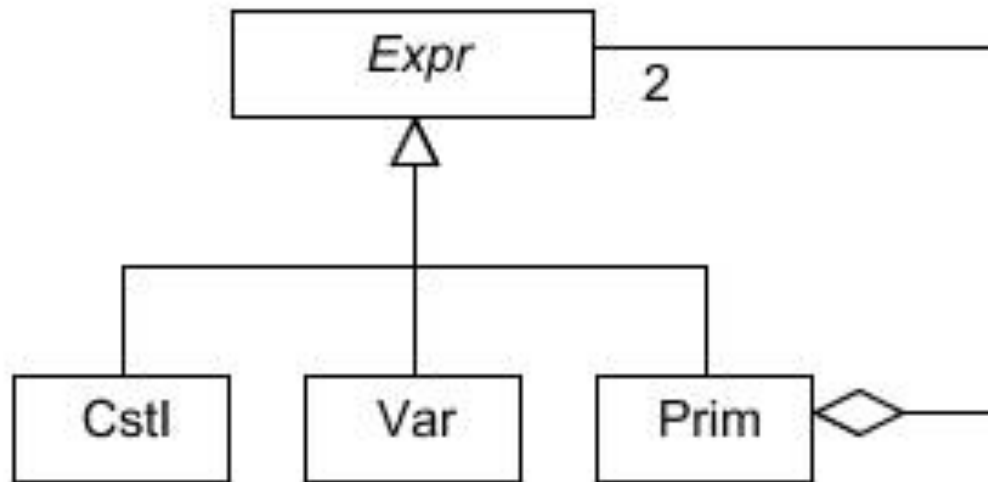
```
let rec eval e (env : (string * int) list) : int =  
  match e with  
  | CstI i           -> i  
  | Var x            -> lookup env x  
  | Prim("+", e1, e2) -> eval e1 env + eval e2 env  
  | Prim("*", e1, e2) -> eval e1 env * eval e2 env  
  | Prim("-", e1, e2) -> eval e1 env - eval e2 env  
  | Prim _           -> failwith "unknown primitive";;
```

Representing abstract syntax in Java

```
type expr =  
  | CstI of int  
  | Var of string  
  | Prim of string * expr * expr;;
```

Functional
style

- Instead of a datatype, use an *abstract class*, *inheritance*, and *composites*:



Object-oriented
style

The expression class declarations

```
abstract class Expr { }
class CstI extends Expr {
    protected final int i;
    public CstI(int i) {
        this.i = i;
    }
}
class Var extends Expr {
    protected final String name;
    public Var(String name) {
        this.name = name;
    }
}
class Prim extends Expr {
    protected final String oper;
    protected final Expr e1, e2;
    public Prim(String oper, Expr e1, Expr e2) {
        this.oper = oper; this.e1 = e1; this.e2 = e2;
    }
}
```

Only fields and
constructors so far

Some expressions

```
Expr e1 = new CstI(17);  
Expr e2 = new Prim("+", new CstI(3), new Var("a"));  
Expr e3 =  
    new Prim("+", new Prim("*", new Var("b"), new CstI(9)),  
            new Var("a"));
```

Evaluating expressions

```
abstract class Expr {  
    abstract public int eval (Map<String,Integer> env) ;  
}  
class CstI extends Expr {  
    protected final int i;  
    public int eval (Map<String,Integer> env) {  
        return i;  
    }  
}  
class Var extends Expr {  
    protected final String name;  
    public int eval (Map<String,Integer> env) {  
        return env.get(name) ;  
    }  
}  
class Prim extends Expr {  
    protected final String oper;  
    protected final Expr e1, e2;  
    public int eval (Map<String,Integer> env) {  
        if (oper.equals("+"))  
            return e1.eval(env) + e2.eval(env) ;  
        else if ...  
    }  
}
```

Abstract eval method

Environment as map
from String to int

Subclasses
override eval

Creating the Environment

```
Map<String,Integer> env0 = new HashMap<String,Integer>();  
    env0.put("a", 3);  
    env0.put("c", 78);  
    env0.put("baf", 666);  
    env0.put("b", 111);
```

- What is the content of the `env0` above?

```
    env0.put("a", 33);  
    env0.put("b", 999);
```

- What is now the content of the `env0` above?
- Are you able to get back the old `env0`?
- Is this like the environment we defined in F#?

Evaluating an expression

- Evaluating an expression given an environment

```
int r1 = e1.eval(env0);
```

- How to format an expression as a `String`?

```
Abstract class Expr {  
    ...  
    abstract public String fmt();  
}  
class Var extends Expr {  
    ...  
    public String fmt() {  
        return name;  
    }  
}
```

See SimpleExpr.java

Semantics for Pretty Printing

- Pretty Printing by substituting variables for their values in the environment:

```
Abstract class Expr {  
    ...  
    abstract public String fmt();  
    abstract public String fmt2 (Map<String,Integer> env) ;  
}  
class Var extends Expr {  
    ...  
    public String fmt2 (Map<String,Integer> env) {  
        return ""+env.get(name) ;  
    }  
}
```

See SimpleExpr.java

```
System.out.println(e1.fmt() + " = " + e1.fmt2(env0) +  
                    " = " + e1.eval(env0)) ;
```

Semantics for Pretty Printing

- Pretty Printing by substituting variables for their values in the environment:

With Programs as Data, we can interpretate the same programs using different semantics.

E.g., `eval`, `fmt` and `fmt2` are all different semantical interpretations of the same program.

`env) ;`

`java`

```
System.out.println(e1.fmt() + " = " + e1.fmt2(env0) +  
                    " = " + e1.eval(env0)) ;
```

Functional vs object-oriented

	Functional	Object-oriented
Expression variant	Datatype constructor	Subclass
Choice in operation	Pattern matching in function	Virtual method in subclasses
Adding a new expression variant	Edit <i>several</i> functions (add new variant to each one)	Add <i>one</i> subclass (with all operations)
Adding a new expression operation	Add <i>one</i> function (operation on all variants)	Edit <i>several</i> classes (add new operation to each one)
Match composite expressions	Easy	Hard



Example: Expression simplification

- $0 + e_2$ gives e_2 ; $e_1 + 0$ gives e_1 ; $1 * e_2$ gives e_2
- Easy with pattern matching:

```
let rec simp e =  
  match e with  
  | Prim("+", CstI 0, e2) -> e2  
  | Prim("+", e1, CstI 0) -> e1  
  | Prim("*", CstI 1, e2) -> e2  
  | ...                    -> ...
```

- Difficult with C++/Java/C#-style single virtual dispatch
- Newer OO languages such as Scala make this easier than Java and C#